Unit II
Image Processing Fundamentals

Image Enhancement

Image enhancement is technique to **improves the visual quality** of an image to make it more **suitable for analysis**.

It does NOT add new information — it enhances existing features.

Type of Enhancement:

Image enhancement approaches fall into two broad categories:

1. **Spatial Domain Method** - This approach **enhancement** is done **directly on pixel value of an image**.
2. **Frequency Domain Method** - This approach **enhancement** is done on **transformed image** (**Fourier transform** of an image)

1. Spatial Domain Method:

Spatial domain enhancement directly manipulates image pixels to improve quality

Mathematical representation:

$$g(x, y) = T[f(x, y)]$$

Where, f(x,y) = input image

g(x,y) = enhanced image

T = Transformation function defined over some neighborhood of (x, y).

Type of spatial Domain Enhancement -

- Point Processing operations
- Histogram Processing
- Spatial Filtering
- Arithmetic/ Logic operation

➢ Point Processing

It operates on individual pixels. **New pixel value is determined solely by its original intensity** based on transfer function.

Mathematical representation :

$$s = T(r)$$

Where, r - original gray value at (x,y)

s - new gray value at (x,y)

Point processing is use to adjusting brightness, contrast, and the emphasis of certain pixel gray levels to improve image visibility and feature extraction

Some common methods of point processing - **Image Negation , Logarithmic Transformation (Log & Inverse log), Power-law (Gamma) Transformation, Contrast Stretching, Thresholding, Intensity Level Slicing , Bit-plane slicing etc**

➢ Histogram Processing

It **improve contrast** by analyzing and **modifying the distribution of pixel intensity values**.

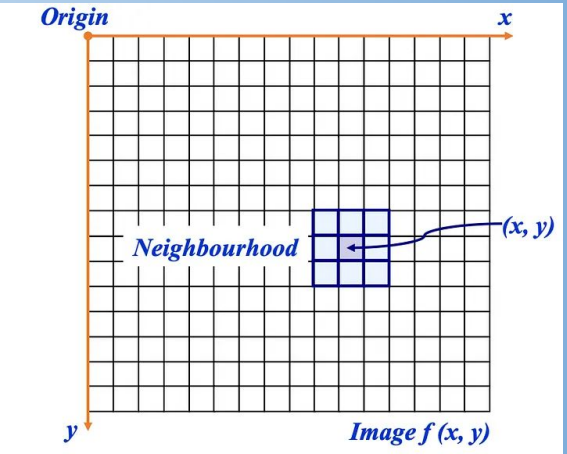
Some common methods of histogram processing - **Histogram equalization etc**

➢ Spatial Filtering

In this method new pixel value is determined by using its **neighboring pixels values and a filter mask** (also called **kernel** or window). Kernel can be of any size form 3x3, 5x5 etc.

Mathematical formula;

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t).\, f(x+s, y+t)$$



Where, f(x,y) = input image

g(x,y) = output image

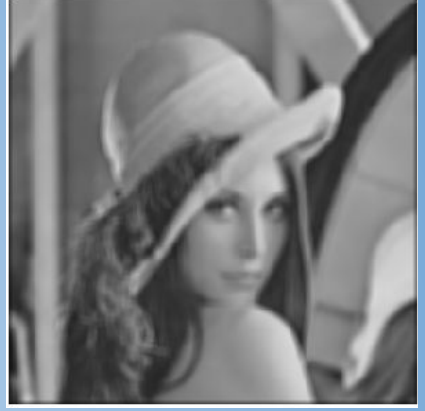
w(s,t) = filter mask coefficients

(2a+1)×(2b+1) = size of the filter mask

Spatial Filtering is used for image enhancement **to get rid of noise, sharpen image** for edge detection and feature extraction in computer vision

Types of Spatial filters:

There are two main categories of spatial filters



- **Smoothing Filters**

  Smoothing filters are **use to blur a image and remove/reduce noise** from images.

  Type of smoothing filters -

  a. **Average (Mean) Filter** - Each pixel is replaced with the **average of its neighbors**. Suitable for image having noise like **Gaussian , uniform**. Edge preservation is poor and sensitive to outliers.
  b. **Median Filter** - Replaces each pixel with the **median of neighborhood values**. Best for removing **salt & pepper noise**. Edge preservation is better and it's not sensitive to outliers.

Example 1: Apply Mean filter of size 3x3 on given image using padding.

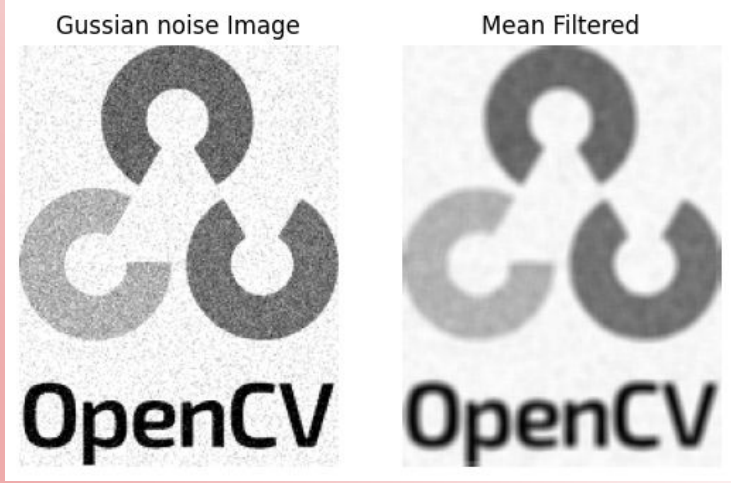
$$\begin{bmatrix} 114 & 133 & 122 \\ 93 & 116 & 95 \\ 123 & 153 & 141 \end{bmatrix}$$

Example 2: Apply Median filter of size 3x3 on given image using padding.

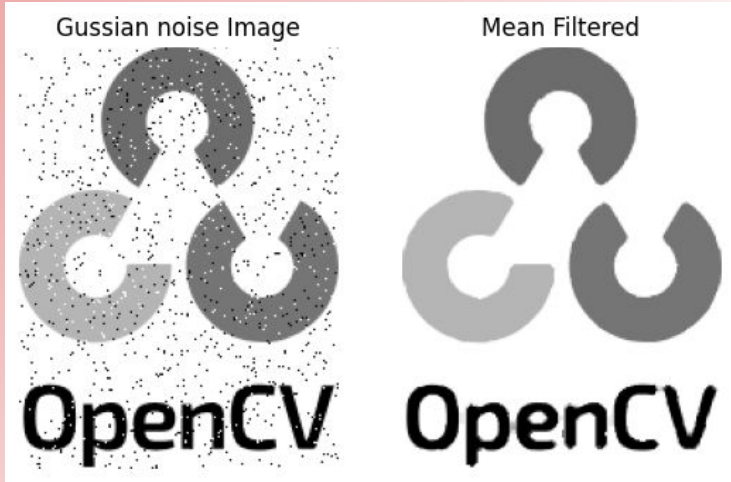
$$\begin{bmatrix} 117 & 117 & 117 \\ 117 & 0 & 117 \\ 1 & 117 & 117 \end{bmatrix}$$

➢ Smoothing Filter:

```
# Mean filtering (3x3)
mean_filtered = cv2.blur(img, (3,3))
```




```
# Median filtering (5x5)
median_filtered = cv2.medianBlur(img, 5)
```



Types of Spatial filters:

- **Sharpening Filters**

  Sharpening filters are used **to enhance blurry image, highlight edges, fine details, texture and boundaries** of image. It **highlights rapid intensity changes** and improves image clarity.

  They are also use to detect edges in image. Edges occur where intensity changes rapidly. Sharpening **uses derivatives**.

  Slow change → small derivative
  Rapid change → large derivative

$$R = w_1 z_1 + w_2 z_2 + \ldots\ldots w_9 z_9$$
$$= \sum_{k=1}^{9} w_k z_k$$

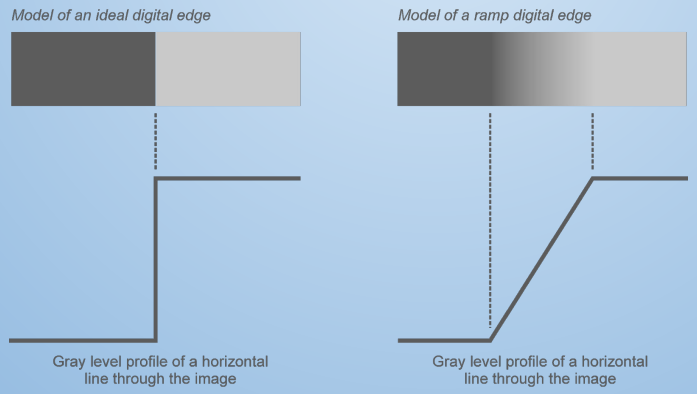
  Type of sharpening filters -

  a. **First-Order Derivative (Gradient Based)** - Sobel, Prewitt, Roberts filter.
  b. **Second-Order Derivative** - Laplacian

Edge Detection:

Edges are often associated with the boundaries of an objects and provide topology and structure information. It consist of meaningful features and contains significant information.

Edges can be detected by detecting sharp/ local changes in intensity, discontinuity in image brightness or contrast. This can be detected by using first and second order derivatives.



Some of the edge detection methods:

- **First-Order Derivative (Gradient method)**

  The gradient measures intensity change in x and y directions.

$$G = \sqrt{(G_x)^2 + (G_y)^2}$$

Where, Gx - change in horizontal direction
Gy - change in vertical direction

Some 1st order derivative operator for edge detection-

a. **Sobel operator -** Use to detect vertical or horizontal line.

Gx = $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ Gy = $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

b. **Prewitt operator -** Use to detect vertical or horizontal line.

Gx = $\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$ Gy = $\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$

b. **Robinson Compass Operator** : It is also known as the **direction mask**. It has eight orientations. It extracts the edges with respect to its direction.

$$NW = \begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix} \quad NE = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix} \quad SW = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -2 \end{bmatrix} \quad SE = \begin{bmatrix} 0 & -1 & -2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}$$

- **Second-Order Derivative (Laplacian method)**

$$\nabla^2 f = \frac{d^2 f}{dx} + \frac{d^2 f}{dy}$$

The Laplacian highlights regions of rapid intensity change.

$$+ve\, Laplacian = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \qquad +ve\, Laplacian = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$Mean\, Laplacian = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Example 1: Detect horizontal & vertical edge in given image. Use kernel size 3x3 & padding.

$$\begin{bmatrix} 1 & 7 & 2 \\ 7 & 7 & 7 \\ 1 & 7 & 3 \end{bmatrix}$$

Example 2: Apply laplacian filter of size 3x3 on given image

$$\begin{bmatrix} 1 & 7 & 2 \\ 7 & 7 & 7 \\ 1 & 7 & 3 \end{bmatrix}$$

➢ Sharpening Filter:

```
# Sobel in X direction
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)

# Sobel in Y direction
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
```

➢ Sharpening Filter:

```
# Robinson Compas
kernelNW = np.array([[-2,-1,0], [-1,0,1], [0,1,2]])
kernelNE = np.array([[0,1,2], [-1,0,1], [-2,-1,0]])
prewittx = cv2.filter2D(img, -1, kernelNW)
```

➢ Sharpening Filter:

```
# Apply Laplacian
laplacian = cv2.Laplacian(img, cv2.CV_64F)
```

Canny Edge Detection Algorithm:

Canny Edge Detection is one of the most powerful and widely used edge detection algorithms in image processing. Its multi-stage detector which uses gradient magnitude & direction. Its robust to noise as it uses gaussian smoothing method.

Algorithm :

1. **Noise removal** - Apply **Gaussian Smoothing**. Edges are highly sensitive to noise.

Mathematical formula of Gaussian filter:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Example, 3x3 Gaussian kernel (sigma=1). This will smooth image and reduces noise.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

2. Gradient Calculation: After smoothing, find intensity change using Sobel operators.

Sobel X (Gx): $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ Sobel Y (Gy): $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

Calculate gradient Magnitude:

$$G = \sqrt{Gx^2 + Gy^2}$$

3. Gradient Direction :

$$\theta = \tan^{-1}\left(\frac{Gx}{Gy}\right)$$

Direction tells orientation of edge.

4. Edge Thinning (Non-Maximum Suppression) :

- Look at gradient direction.
- Compare pixel with neighbors in that direction.
- Keep pixel only if it is maximum.
- Otherwise set to 0.

Example:

If gradient direction is 90° (vertical), compare top and bottom pixels.

If center is not greater → suppress it.

5. Double Thresholding : To reduce false edges.

Use two thresholds:

- High threshold (TH)
- Low threshold (TL)

Classify pixels:

- If $G > TH \rightarrow$ Strong edge
- If $TL < G < TH \rightarrow$ Weak edge
- If $G < TL \rightarrow$ Non-edge

6. Edge Tracking by Hysteresis:

Weak edges connected to strong edges → Keep them

Weak edges not connected → Remove

Example: Detect edge at center pixel (50)

$$I = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 100 \\ 10 & 100 & 150 \end{bmatrix}$$

Step 1: Apply Gaussian smoothing filter

$$G * I$$

$$= \frac{1}{16}((1 \times 10) + (2 \times 10) + (1 \times 10) + (2 \times 10) + (4 \times 50) + (2 \times 100) + (1 \times 10) + (2 \times 100) + (1 \times 150))$$
$$= 51.25 \approx 51$$

Step 2: Calculate Gradient (Sobel X & Y) & magnitude

Gx = 320 Gy = 320

$$G = \sqrt{Gx^2 + Gy^2}$$
$$= \sqrt{320^2 + 320^2}$$
$$= 452.55 \approx 453$$

Step 3: Calculate Gradient Magnitude

$$\theta = \tan^{-1}\left(\frac{Gy}{Gx}\right)$$
$$= \tan^{-1}\left(\frac{320}{320}\right)$$
$$= 45^\circ$$

Edge direction **diagonal**

Step 2: Non-Maximum Suppression

- Compare center pixel with diagonal neighbors.

  Diagonal neighbors: Top-left (10), Bottom-right (150)

- Since gradient magnitude (453) is highest in neighborhood, Keep pixel.
- If it was smaller → set to 0.

Step 3: Double Thresholding

Assume:

High Threshold = 400
Low Threshold = 200

Gradient = 453

Since, 453>400 → Strong edge

Step 4: Hysteresis

Strong edges are directly kept. Final output pixel = Edge (1 or 255)

➢ Canny Edge Detection:

```
edges = cv2.Canny(img, 100, 200)
```

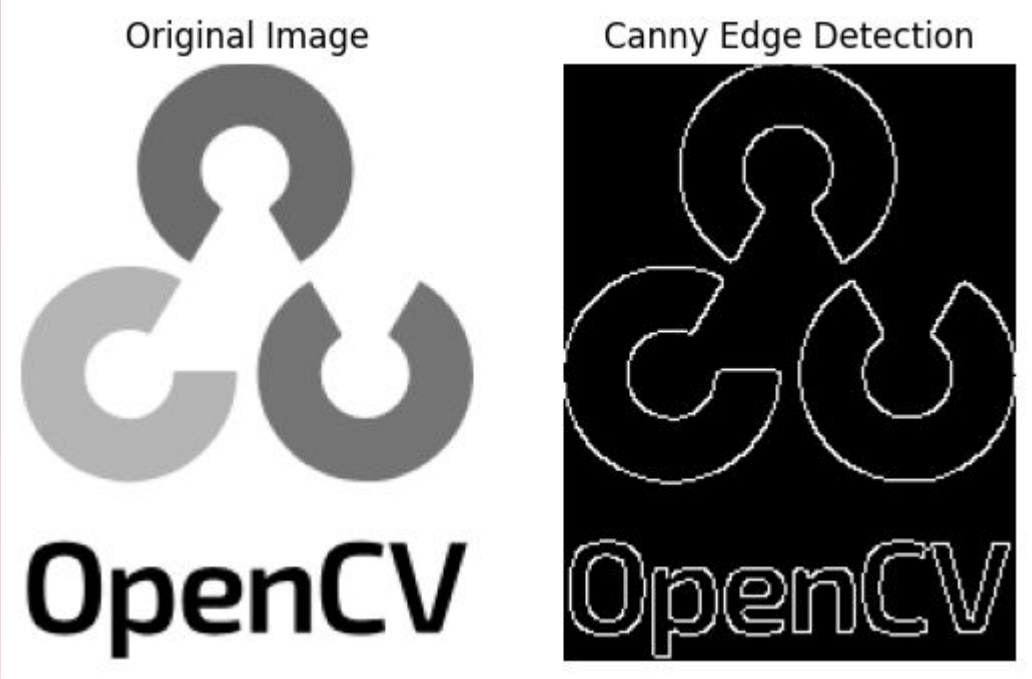


Image Transformations:

Image transformations **convert an image from one domain to another** to make analysis easier.

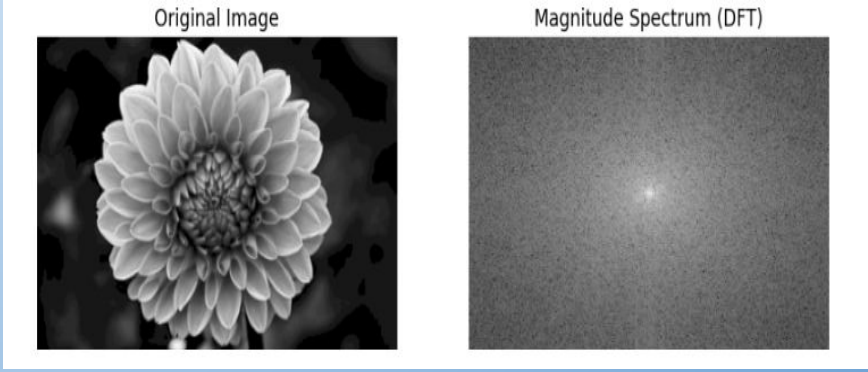
1. **Fourier Transform -**

The Fourier Transform converts an image from:

Spatial Domain (pixels) → Frequency Domain (frequencies)

It helps analyze:

- High frequencies → edges, noise
- Low frequencies → smooth regions



To transform image into frequency domain **Fourier Transform (FT)** of image is computed first.

2D Discrete Fourier Transform :

$$F(u,v) = \sum_{N=0}^{N-1} \sum_{M=0}^{M-1} f(x,y) e^{-2\pi j\left(\frac{xu}{M}+\frac{yv}{N}\right)}$$

Where, f(x,y) - spatial image

F(u,v) - frequency component

M,N - size of image

Magnitude Spectrum:

$$|F(u,v)| = \sqrt{Re^2 + Im^2}$$

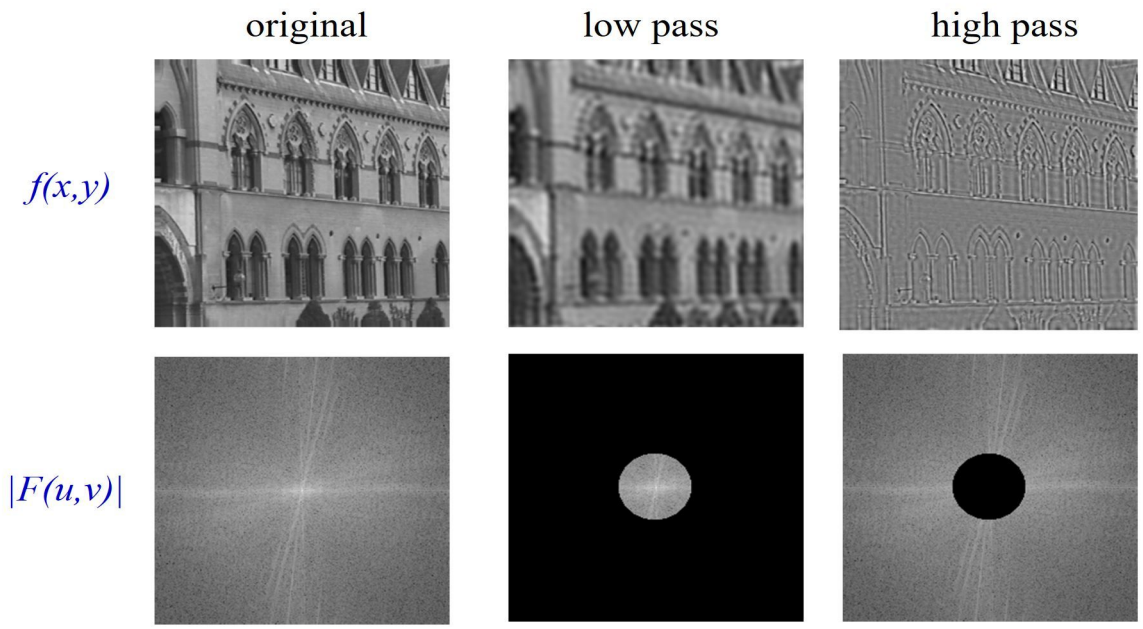
Usually we display: Magnitude (dB) = 20 $\log_{10}$ (1+|F|)

Image Enhancement like smoothing and sharpening can also be done in **Frequency domain by removal of high or low frequency components.**

Frequency domain filters are different from spatial domain filters as it basically focuses on the frequency of the images.

There are of 3 types:

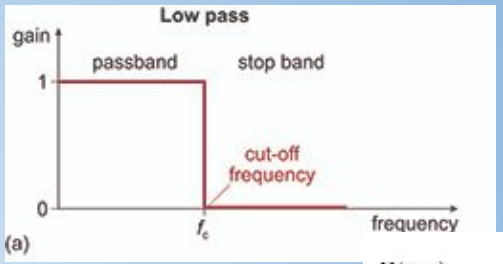
- Low pass filter
- High pass filter
- Band pass filter



1. **Low Pass Filter** : It is **used to smoothen** the image **by attenuating high frequency** components (sharp edges, noise) **and preserving low frequency** components (smooth part like background, sky etc)

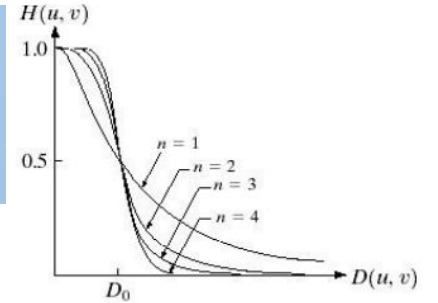
- **Ideal Low Pass Filter (ILPF):**

$$H(u,v)=\begin{cases}1 \text{ if } D(u,v)\le D_0\\ 0 \text{ if } D(u,v)>D_0\end{cases}$$



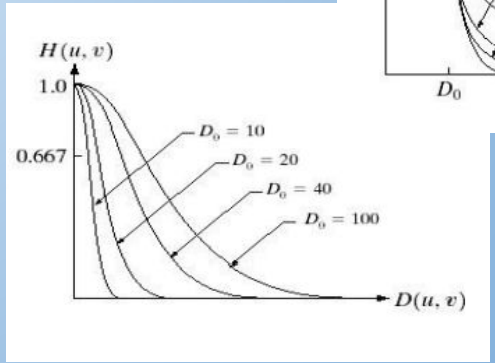
- **Butterworth Low Pass Filter (BLPF):**

$$H(u,v)=\frac{1}{1+[D(u,v)/D_0]^{2n}}$$



- **Gaussian Low Pass Filter:**

$$H(u,v)=e^{-D^2(u,v)/2D_0^2}$$



2. **High Pass Filter** : It is **used to sharpening** the image **by attenuating low frequency** components (smooth part like background, sky etc) **and preserving high frequency** components (sharp edges, noise)

High pass filters are precisely the reverse of low pass filters.

$$H_{hp}(u, v) = 1 - H_{lp}(u, v)$$

Refer : Unit 3 - Image Enhancement in Frequency domain (All slides)

Example: Calculate |F(u,v)| of give image

$$F(u,v) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Image Transformations:

2. Hough Transform -

The Hough Transform converts an image from:

Image space → Parameter space

It used for:

- Detecting lines
- Detecting circles
- Shape detection

It is mainly used after edge detection (usually using Canny edge detection). In real images edges may be broken, presence of noise so normal methods get failed. But Hough transform can detect incomplete lines, noisy shapes accurately.

➢ Fourier transformation:

```
# Apply DFT
dft = np.fft.fft2(img)
dft_shift = np.fft.fftshift(dft)

# Compute magnitude spectrum
magnitude = 20 * np.log(np.abs(dft_shift))
```

Morphological Operation:

Morphological operations are non-linear image processing **techniques based on shape**, using a small template called a **structuring element**.

It is used on **binary and grayscale images** for noise removal, object isolation, and boundary detection, they apply **structuring elements** to define, alter, or analyze shape, size, and structure.

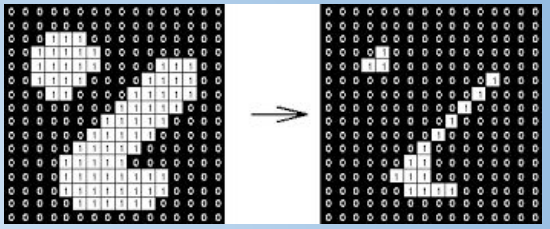
Structuring Element is small matrix or kernel that defines the neighborhood of pixels over which the operation is performed.

Some morphological operations are -

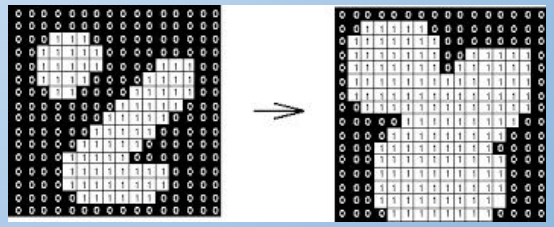
- Erosion
- Dilation
- Opening
- Closing etc

Note: Always consider zero padding while performing morphological operations.

1. **Erosion (A⊖B)** : Erosion is a fundamental morphological operation that **reduces the size of objects** in a binary image. It works by removing pixels from the boundaries of objects.
   - Output pixel = 1
   - Only if all 9 pixels under kernel = 1

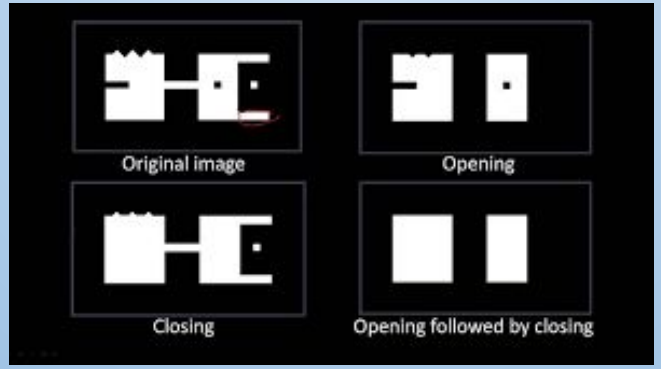


2. **Dilation (A⊕B)** : Dilation is the opposite of erosion and is used to increase the size of objects in an image. To join adjacent objects, fill small holes, and enlarges objects.
   - Output pixel = 1
   - If any one pixel under kernel = 1



3. **Opening $A \circ B = (A \ominus B) \oplus B$** : Opening is a compound operation that involves **erosion followed by dilation**. First, the image undergoes erosion, which **removes small objects and noise**. Then, dilation is applied to restore the size of the remaining objects to their original dimensions.

4. **Closing $A \cdot B = (A \oplus B) \ominus B$** : Closing is another compound operation that consists of **dilation followed by erosion**.First, dilation is applied to the image, **filling small holes and gaps**. Then, erosion is applied to restore the original size of the objects.



Example 1 : Perform Erosion, Dilation, Opening & closing on given image

$$A = \begin{matrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{matrix} \qquad B = \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix}$$

Example 2: Perform Opening & closing (Noise Removal) on given image.

$$A = \begin{matrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{matrix} \qquad B = \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix}$$

Example 1 : Perform closing on given image. Does it fill the hole? Perform Opening and compare results

$$A = \begin{matrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{matrix} \qquad B = \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix}$$

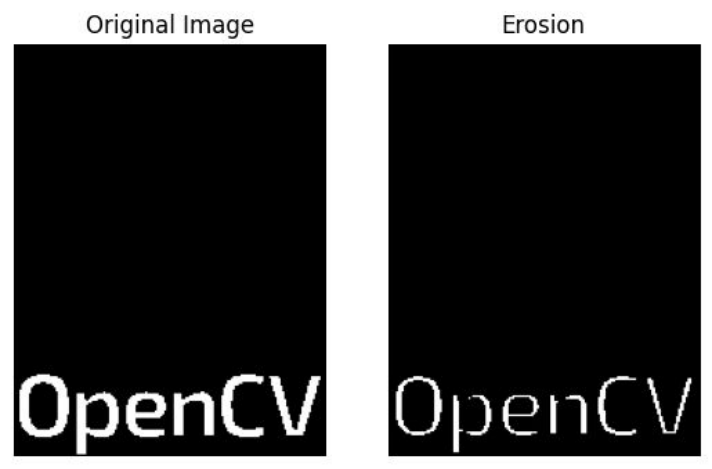
Example 2: Perform **A⊖B , A⊕B, (A⊖B)⊕B, (A⊕B)⊖B** on given image.

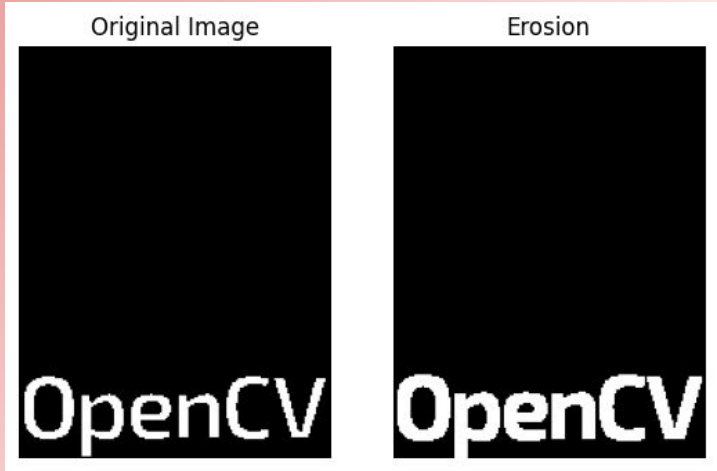
$$A = \begin{matrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{matrix} \qquad B = \begin{matrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{matrix}$$

➢ Morphological Operation:

- Erosion :

```
kernel = np.ones((3,3), np.uint8)
# Erosion
erosion = cv2.erode(img, kernel, iterations=1)
# dilation
dilation = cv2.dilate(img, kernel, iterations=1)
```

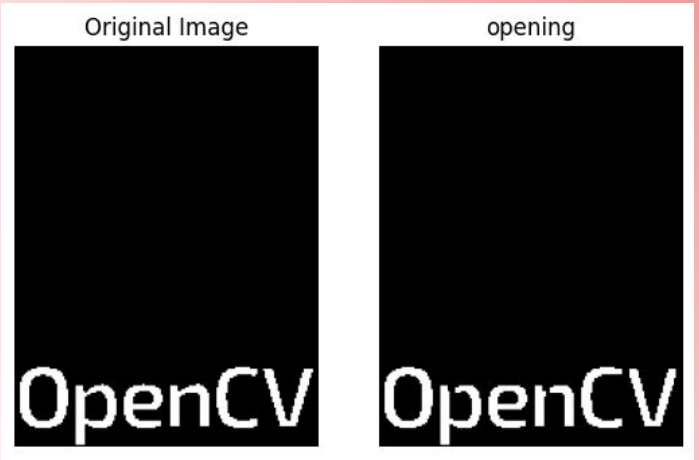


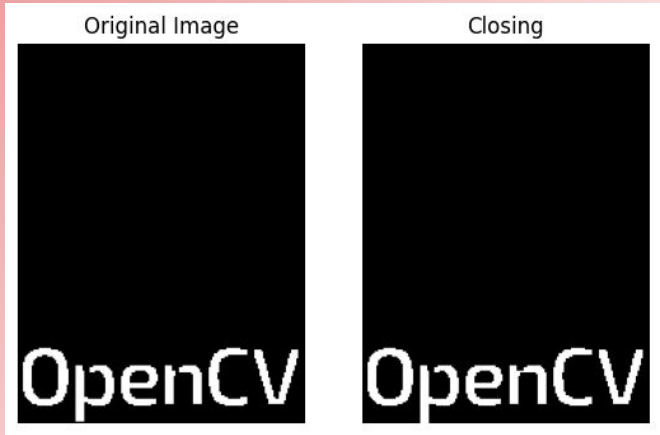


➢ Morphological Operation:

- Erosion :

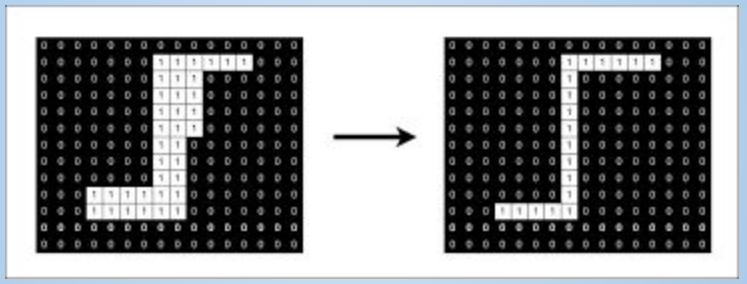
```
kernel = np.ones((3,3), np.uint8)
#Opening
opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
#closing
closing = cv2.morphologyEx(opening, cv2.MORPH_CLOSE, kernel)
```





5. Skeletonization: Skeletonization reduces a binary object to a thin version (1-pixel wide) that preserves shape, connectivity and topology of object.

The result is called the skeleton or medial axis of the object. It is widely used in OCR, fingerprint analysis, shape recognition, and medical image analysis.



Assignment 2 (Theory) Due Date: 16/03/2025

1. What are different image enhancement techniques in spatial domain?
2. Explain spatial filtering technique.
3. Apply mean filter of size 3x3 on given image and compute output image (use padding)

$$\begin{bmatrix} 114 & 133 & 122 \\ 93 & 116 & 95 \\ 123 & 153 & 141 \end{bmatrix}$$

4. Apply median filter of size 3x3 on given image on given image using padding (use padding)

$$\begin{bmatrix} 117 & 117 & 117 \\ 117 & 0 & 117 \\ 1 & 117 & 117 \end{bmatrix}$$

Assignment 2 (Theory)

5. Write first-order derivative (gradient based) & second order derivative operators.
6. Explain Edge detection algorithm?
7. Define Image transformation.
8. Write notes on - Image enhancement in frequency domain.
9. Write short notes on - Erosion, Dilation, Opening & Closing
10. Explain - erode( ) , dilate( ), morphologyEx( ), filter2D( ), blur( ), medianBlur( )
11. Perform all numerical on morphological operation.

Assignment (Practical)

- Write python function **myfilter( )** for all types of spatial domain filtering.
- Design Canny edge detection function.
- Design low pass filter, high pass filter for NxM image